Разбор статьи “Training Transformers with 4-bit Integers”

Подготовил Соколов Илья

Цель статьи

Показать, как перевести все matmul в INT4 при обучении и как получить от этого профит.

Промежуточные незатратные на compute вычисления проходили в fp16

Основные этапы

Авторы разделили процесс квантизации на 2 этапа:

1. Квантизация на forward
2. Квантизация на bacward

Введение

Авторы подчеркивают, что сама по себе INT4 квантизация сложна, но попытки сообщества были.

Предыдущие результаты: для ускорения прошлые методы не подходили, так как требовали custom формата. Поэтому прогресс был именно в снижении размерности.

Forward

Основная идея:
Авторы попробовали LSQ - стандартную технику QAT, но из-за выбросов (outliers) качество очень сильно проседало

Методы ранее не концентрировались на PTQ и QAT с INT4

Что делать с outliers?

Квантизатор Адамара - Hadamard quantizer (HQ)

В чем смысл? Авторы заявляют, что только несколько колонок из всей матрицы содержат выбросы.

Давайте переведем матрицы через преобразование Адамара в такое пространство, которое спроецирует выбросы в другие сущности (значения)

Алгоритм квантизатора Адамара

Определяем матрицы весов и активаций:

$$\mathbf{X} = (\mathbf{XH})\mathbf{H}^\top \approx s_X \text{int}_{s_X}(\mathbf{XH})\mathbf{H}^\top, \qquad \mathbf{W} = (\mathbf{WH})\mathbf{H}^\top \approx s_W \text{int}_{s_W}(\mathbf{WH})\mathbf{H}^\top.$$

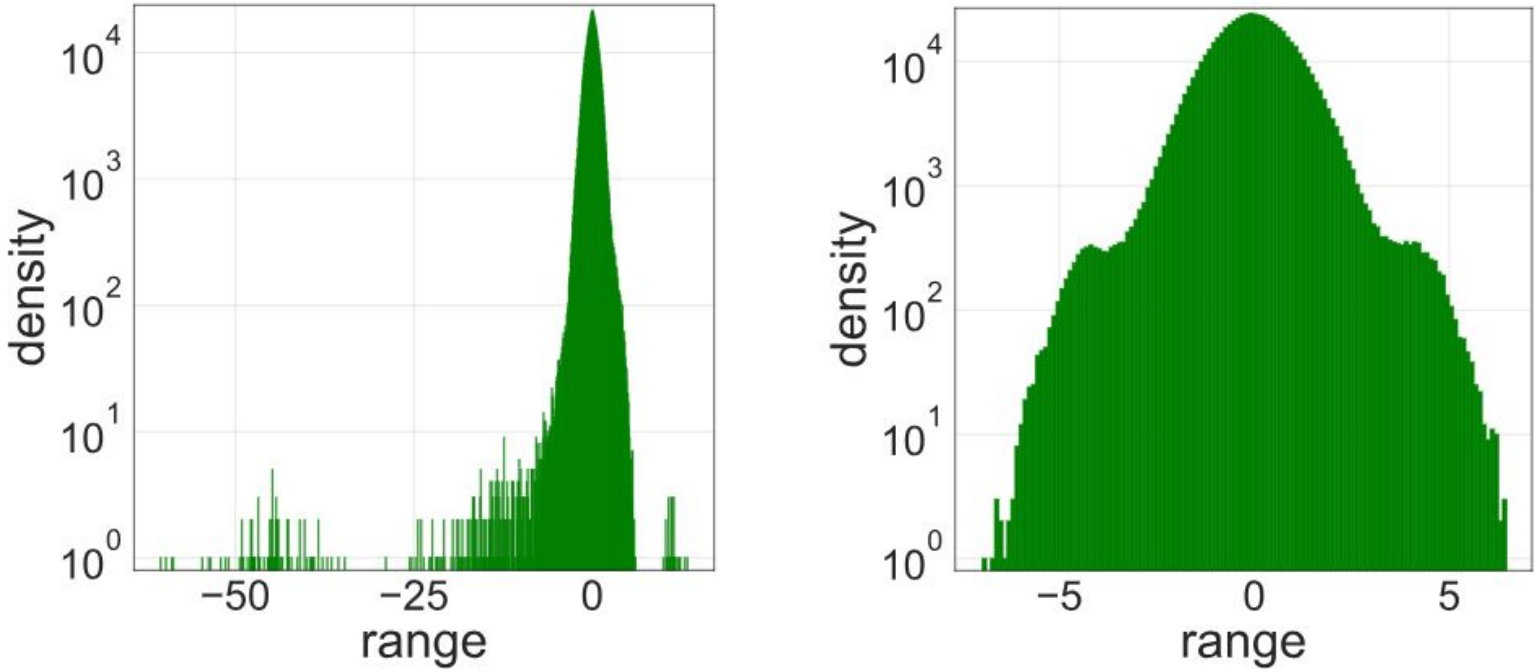
Подставляем в выражение:

$$\mathbf{Y} = \mathbf{XW}^\top \approx s_X s_W \text{int}_{s_X}(\mathbf{XH})\mathbf{H}^\top \mathbf{H} \text{int}_{s_W}\left(\mathbf{H}^\top \mathbf{W}^\top\right) = s_X s_W \text{int}_{s_X}(\mathbf{XH}) \text{int}_{s_W}\left(\mathbf{H}^\top \mathbf{W}^\top\right)$$

1. Подсчитать XH и H.T @ W.T в fp16
2. С помощью LSQ квантизовать матрицы в INT4
3. Деквантизовать INT32 в fp16 умножанием на scale_x * scale_w

Результаты преобразования

Активация linear-1-2 BERT-base-uncased model



До и после преобразования

Backward

С помощью Straight Through Estimator (STE):

grad_W = scale_x * (grad_y.T @ XH * clamp(X / scale_x, -Qn, Qp))@ H.T

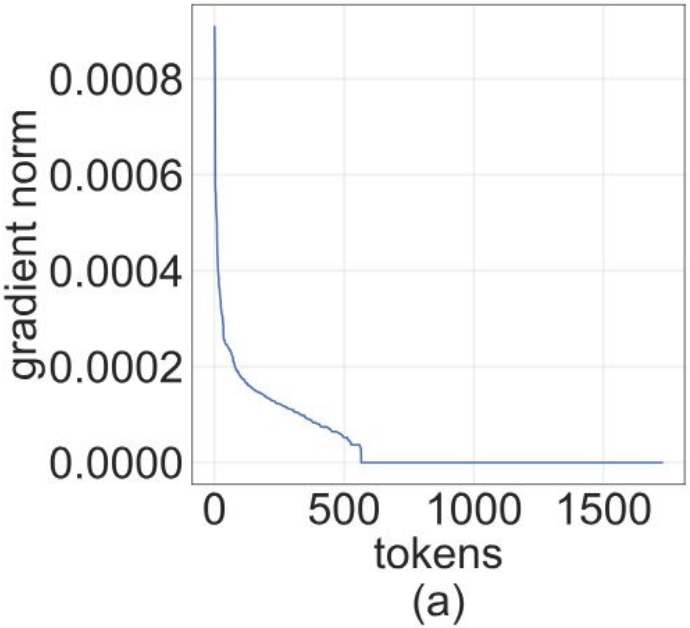
grad_X = scale_w * clamp(W/scale_w , -Qn, Qp) * grad_y) @ WH @ H.T

* - это element_wise умножение

Что нужно знать о градиентах

Матрица градиентов grad_y очень разрежена.

Минус - Overparametrization



Много околонулевых по норме градиентов

Как воспользоваться overparametrization?

Большая часть строк градиента околонулевые, а значит практически не вносят вклада в обновление градиента. В это же время важные градиенты не всегда могут влезть по норме в INT4

Решение? Bit splitting + Leverage Score Sampling

Bit Splitting

Разделим fp матрицу на higher + lower 4 bits:

grad_y = scale_higher * grad_y_higher + scale_lower* grad_y_lower

Можно представить это как INT8 матрицу.

По сравнению с наивной конвертацией grad_y в INT4 здесь в 2 раза больше matmul операций. Исправляем с помощью Leverage Score Sampling

Leverage Score Sampling

Подсчитывает Leverage Score c := l2_norm(grad_yi) * l2_norm(Xi) в fp16

Сэмплируем маски Mi

Сэмплируем строки grad_y и X с помощью Mi

Считаем INT4 маскированные grad_y_higher.T*M_higher*X + grad_y_lower.T*M_lower*X

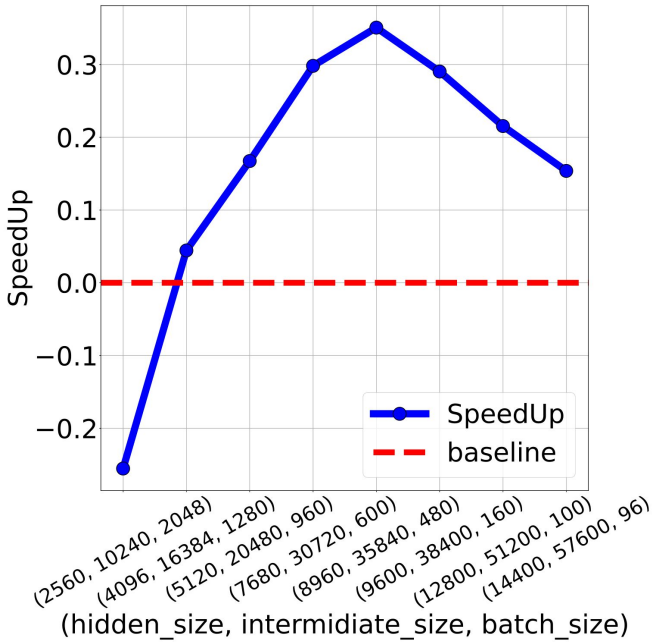
Деквантизуем и суммируем INT32, получаем fp16

Результаты

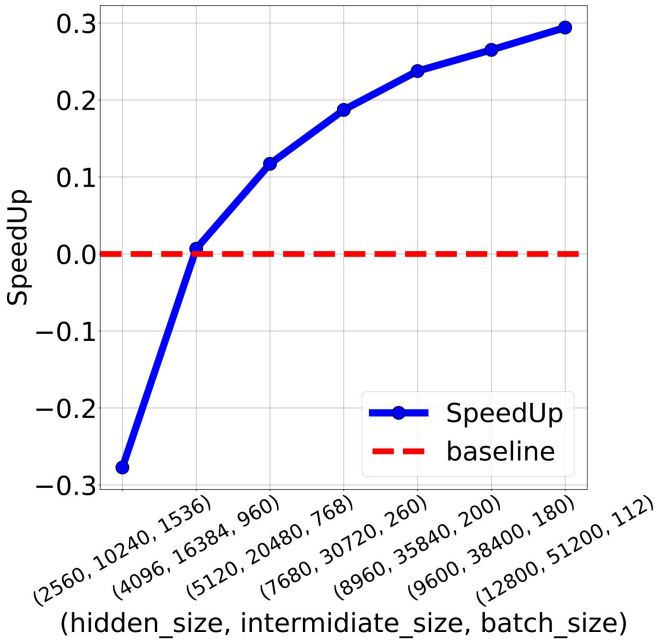
				Baselines		4-bit training methods	
Dataset	Train type	Model	Metric name	FP	INT8	LSQ+LUQ	HQ+LSS
GLUE-dev	FT	Bert-base	Avg	$82.67_{0.24}$	$81.45_{0.13}$	$75.29_{0.52}$	$80.81_{0.31}$
		Bert-large	Avg	$84.57_{0.42}$	$82.74_{0.24}$	$55.93_{2.47}$	$82.25_{0.58}$
SQUAD v1	FT	Bert-base	F1	$88.32_{0.30}$	$88.42_{0.20}$	$85.75_{0.31}$	$87.60_{0.25}$
SQUAD v2	FT	Bert-base	F1	$76.04_{0.68}$	$75.63_{0.07}$	$71.02_{0.41}$	$74.63_{0.18}$
Adversarial QA	FT	Bert-base	F1	$40.99_{0.38}$	$40.17_{0.58}$	$31.85_{0.30}$	$38.70_{0.77}$
SWAG	FT	Bert-base	Acc	$79.84_{0.10}$	$79.18_{0.19}$	$70.79_{1.20}$	$77.49_{0.16}$
CONLL	FT	Bert-base	Acc	$93.38_{0.08}$	$93.13_{0.14}$	$87.63_{0.39}$	$91.90_{0.48}$
WMT	PT	Transformer-base	BLEU	27.5	25.4(Ultra Low)	27.17	-
			SacreBLEU	26.5	-	-	25.57
CIFAR10	FT	ViT-B/32	Top1 Acc	$98.77_{0.03}$	$98.59_{0.02}$	$97.76_{0.10}$	$98.36_{0.05}$
		ViT-L/32		98.98	98.76	98.38	**98.47**
CIFAR100	FT	ViT-B/32	Top1 Acc	$91.94_{0.11}$	$90.99_{0.07}$	$88.63_{0.085}$	$89.78_{0.06}$
		ViT-L/32		93.07	92.2	90.97	**91.13**
ImageNet1k	FT	ViT-B/32	Top1 Acc	81.88	80.42	77.25	**79.18**
		ViT-L/32		81.62	81.3	77.41	**80.06**
		ViT-L/16		84.55	83.05	82.4	**82.61**
	PT	Deit-small	Top1 Acc	73.1	70.95	**69.96**	69.18

Таблица экспериментов из статьи

Результаты. Скорость. Сравнение с fp16



Bert Large



Gpt2-base

А что по скорости?

Заявляется, что квантизатор Адамара быстрее fp16 над matmul операциями. При этом поиск оптимального размера k матрицы H осуществляется перебором по сетке (авторы пришли к k=5) через MSE(X, X_HQ) * MSE(W, W_HQ)

Bit splitting + Leverage Score Sampling практически не замедляют

Воспроизведение результатов

Метрика	INT4	Без квантизации (FP16)
Итоговый Matthews correlation (MCC)	0.510582	**0.595586**
Training time, с	65.8036	**13.0785**
FLOPs	110,098,327	109,483,778
FLOPs learn	109,483,927	109,483,778
Validation loss по эпохам	0.46338; 0.51297; 0.52690	0.41355; 0.50706; 0.50851

По скорости

Представление весов	Расчётный размер
FP16, 2 байта на параметр	208.82 MiB
Упакованный INT4, 0.5 байта на параметр	52.21 MiB

По памяти